

The SVM Kernel Trick: How Curved Boundaries Are Learned Without Explicit Curved Features

Tutorial format: DOCX report, ready to export as PDF if required by CodeGrade

GitHub repository: <https://github.com/gopikapk10-alt/Machine-Learning-Tutorial>

Table of Contents

Learning aim	3
Teaching data: a visible curved problem	3
The SVM idea before kernels	3
How the kernel trick works.....	4
Kernel comparison in code	5
Understanding C and gamma.....	5
Choosing hyperparameters responsibly	6
Practical checklist and responsible use	7
Conclusion	7
References.....	8

Table of Figures

Figure 1: Two-moons data. Colour and marker shape are both used so the figure remains readable for colour-blind readers.....	3
Figure 2: Linear, polynomial and RBF SVM decision boundaries on the same training data.	5
Figure 3: RBF SVM sensitivity. Higher C and gamma can create more flexible boundaries, but excessive flexibility risks overfitting.	6
Figure 4: Cross-validation heatmap for RBF SVM hyperparameters. The selected setting is circled.	6

Table of Table

Table 1:Model Test Accuracy.....	5
----------------------------------	---

Learning aim

Support Vectors Machines (SVMs) are usually introduced and learned as "straight-line classifiers": compute the largest possible margin between two classes and classify new examples based on that margin. The concept is good, but it seems to not work if the true boundary is curved. This tutorial examines how an SVM can learn curved boundaries and yet still work on a maximum-margin problem, via the kernel trick (Ghojogh et al., 2021).

At the end, the reader should be able to cite the reasons why a linear SVM performs poorly on curved data, and why a kernel substitutes a dot product by a similarity function; why the RBF kernel can model local structure; and how to tune its parameters C and γ via cross-validation (Wainer and Fonseca, 2021).

Teaching data: a visible curved problem

The data set is synthetic and mimics two-moons data. This is not an intended real world benchmark; this is a set of data used for teaching because the correct pattern is apparent. The two classes are wrapped around each other in such a way that a straight line can only intersect the shape instead of following it. The entire code that produces the data, trains the models and reproduces the figures is given in the notebook.

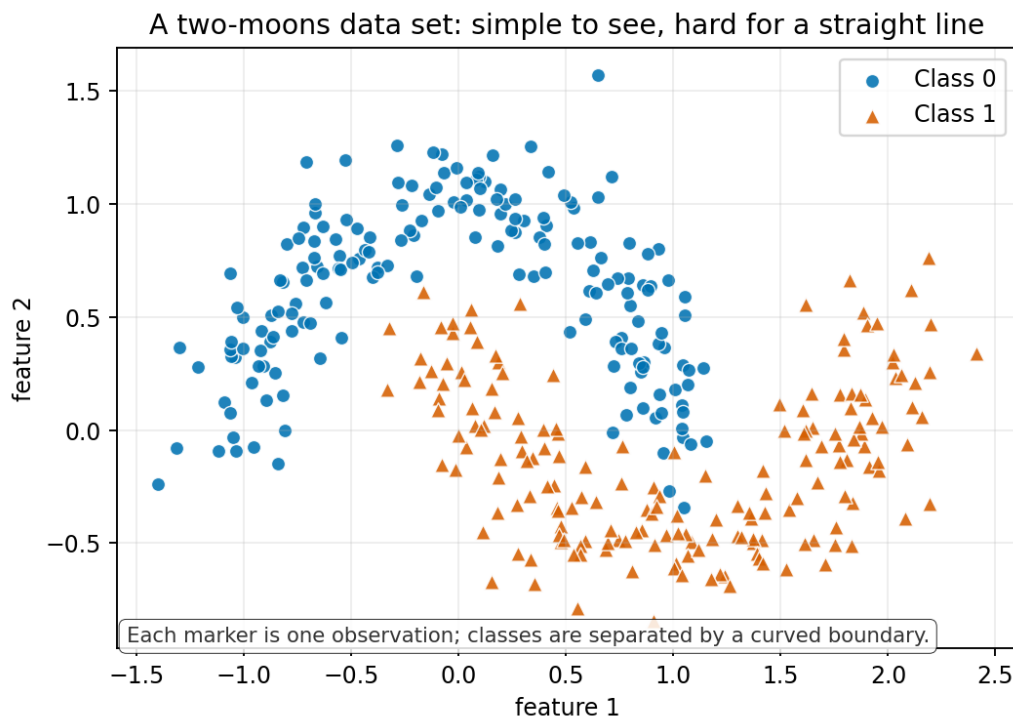


Figure 1: Two-moons data. Colour and marker shape are both used so the figure remains readable for colour-blind readers.

The SVM idea before kernels

A linear SVM is looking for a separating hyperplane, $w^T x + b = 0$. In a soft-margin SVM the desired goal is to seek a large margin while minimizing the cost of errors. It is also interested in an informal value for $\|w\|$ (small), since this leaves a large margin, but it is also interested in the cost of training errors, which is controlled by the value of C . A larger C will give more severe penalties for errors and can make a less "smooth" model (Wang, 2022).

$$\text{minimise } 1/2 \|w\|^2 + C \sum \xi_i$$

$$\text{subject to } y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

This form highlights the margin, and the kernel trick is more obvious in the dual form of the SVM. The last prediction of a new point x can be represented as a weighted sum of similarities between x and the training points in the dual:

$$f(x) = \text{sign}(\sum \alpha_i y_i (x_i^T x) + b)$$

Some of the α_i values are not zero. The support vectors are those important observations. The important thing to note is that the training data are presented via dot products like $x_i^T x$. If one is able to find a better similarity measure than these dot products, then the SVM can act as though the data were mapped into a richer feature space.

How the kernel trick works

The secret is actually not magic, but a computing shortcut. Suppose that each input x is represented as a new feature representation $\phi(x)$. In a new space, the SVM boundary could be straight, although it appears to bend in the original space. One direct way is to compute each transformed feature in the function $\phi(x)$, but when the feature space is very large and/or complex, this may be costly or even impractical.

A kernel does not do the explicit transformation, but instead calculates the inner product directly in the hidden feature space.

$$K(x, z) = \phi(x)^T \phi(z)$$

One can then use the SVM algorithm to substitute each occurrence of $x_i^T x_j$ for $K(x_i, x_j)$. The model imparts similarities, instead of manually building the transformed features, to the model trains and predicts. A valid kernel is symmetric and positive semi-definite, which ensures that it behaves like an inner product in some feature space (Ghojogh et al., 2021).

Here's an example with a small polynomial. The degree-2 polynomial kernel is:

$$K(x, z) = (x^T z)^2 = (ac + bd)^2 = a^2c^2 + 2abcd + b^2d^2$$

This is exactly the dot product of transformed vectors $\phi(x) = (a^2, \sqrt{2}ab, b^2)$ and $\phi(z) = (c^2, \sqrt{2}cd, d^2)$. Then a linear separator in ϕ -space represents a curved boundary in the original 2-dimensional plot. Non-linear power is fed into the model without any need to hand-create a^2 , ab and b^2 features.

What's even better is the Radial Basis Function (RBF) kernel:

$$K(x, z) = \exp(-\gamma \|x - z\|^2)$$

It shows local similarity. The points that are close together have a kernel value that is close to 1; points that are far apart have a kernel value close to 0. One way to think of the RBF kernel is as an implicit infinite dimensional feature map. Prediction can be viewed as a vote from support vectors whose similarity is weighted. Decision score is increased if new observation most similar to positive-class support vectors, else it's decreased if most similar to negative-class support vectors. This is why the RBF boundary can be bent around the moons without making a bend an explicit graph.

Kernel comparison in code

The implementation is based on a scikit-learn Pipeline that includes both SVC and StandardScaler (Pedregosa et al., 2011). Scaling is crucial since the margins used in SVM and RBF distances involve feature scale. To draw a fair comparison, the same training and testing split is used for all kernels.

```
model = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(kernel="rbf", C=1.0, gamma=1.0))
])
model.fit(X_train, y_train)
```

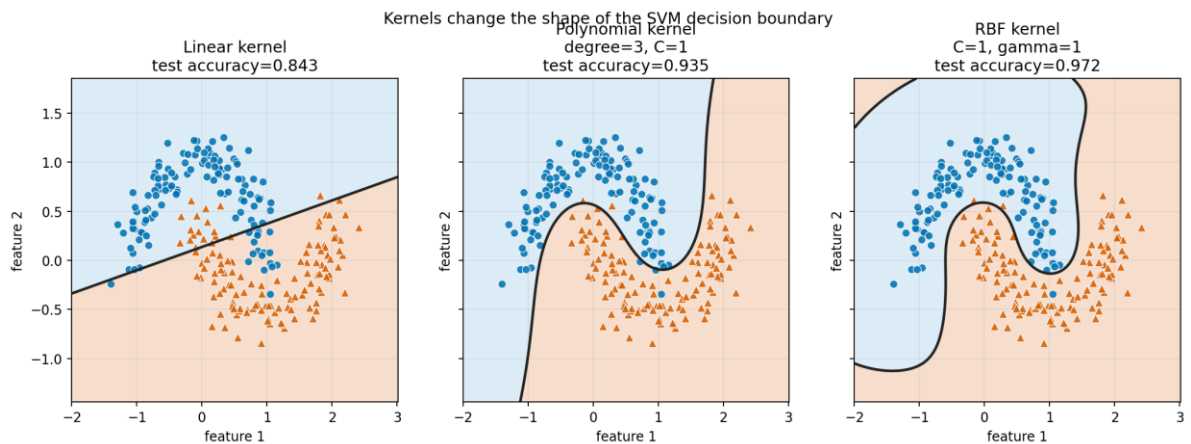


Figure 2: Linear, polynomial and RBF SVM decision boundaries on the same training data.

The linear SVM reached 0.843 test accuracy. It finds a reasonable straight cut but cannot follow the moon shape. The degree-3 polynomial kernel improved to 0.935 by allowing a curved boundary. The RBF kernel reached 0.972 because its local similarity measure matches the local curved structure of the data.

Model	Test accuracy
Linear SVM	0.843
Polynomial SVM, degree 3	0.935
RBF SVM, C=1, gamma=1	0.972

Table 1: Model Test Accuracy

Understanding C and gamma

The RBF SVM is a very strong tool but has a strong dependence on C and gamma. C determines the strength of the model's punishment for its training mistakes. A small C will be more tolerant of a broader and more even margin, though there may be some points that are misclassified. If the C is big, then it is trying to categorize the training data more rigorously, which can make the boundary more complex (Wainer and Fonseca, 2021).

Gamma controls locality. If gamma is low, then the support vectors are able to affect a large area, which makes the boundary smoother. High gamma causes influence to become narrow and the boundary

can bounce around the examples. So, high training accuracy is not sufficient proof that the model is good as it could just be memorising the training set.

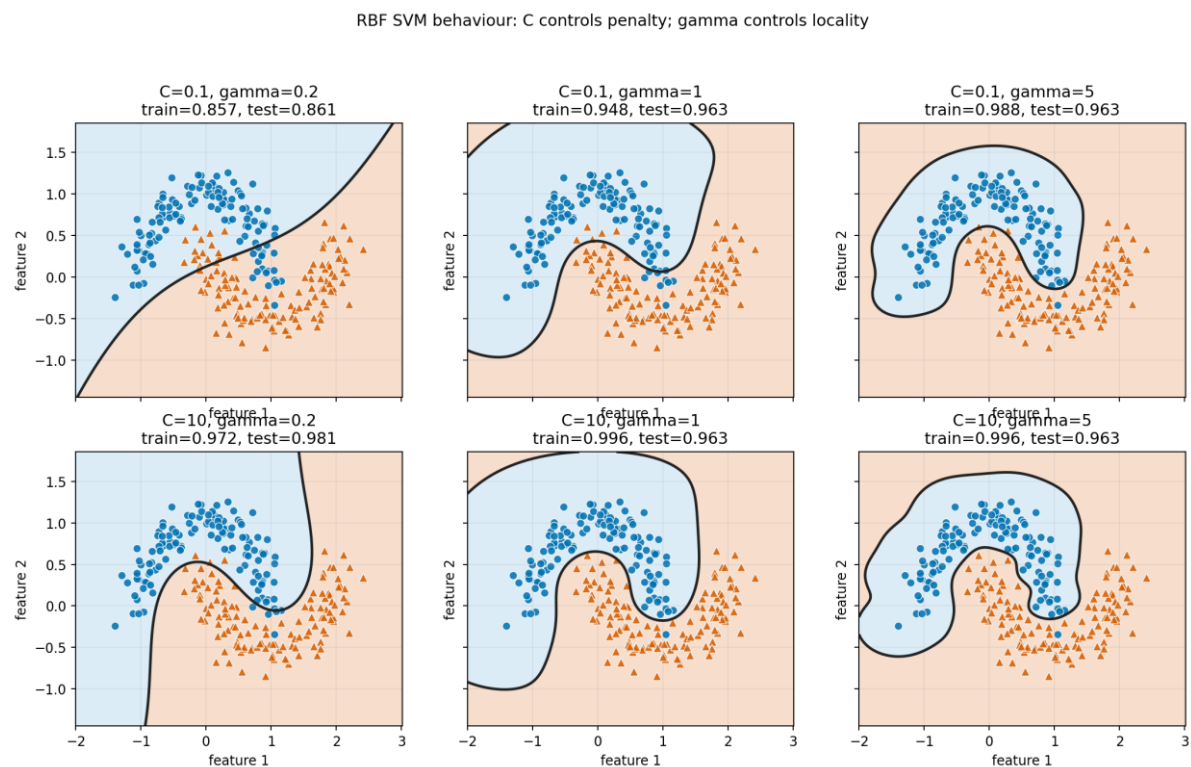


Figure 3: RBF SVM sensitivity. Higher C and γ can create more flexible boundaries, but excessive flexibility risks overfitting.

Choosing hyperparameters responsibly

The RBF hyperparameters are selected by a 5-fold cross validation of the training data. The grid search is performed on the parameters C and γ and a single evaluation of the chosen model is made on the held-out test set. This eliminates the need to tune directly on the test set.

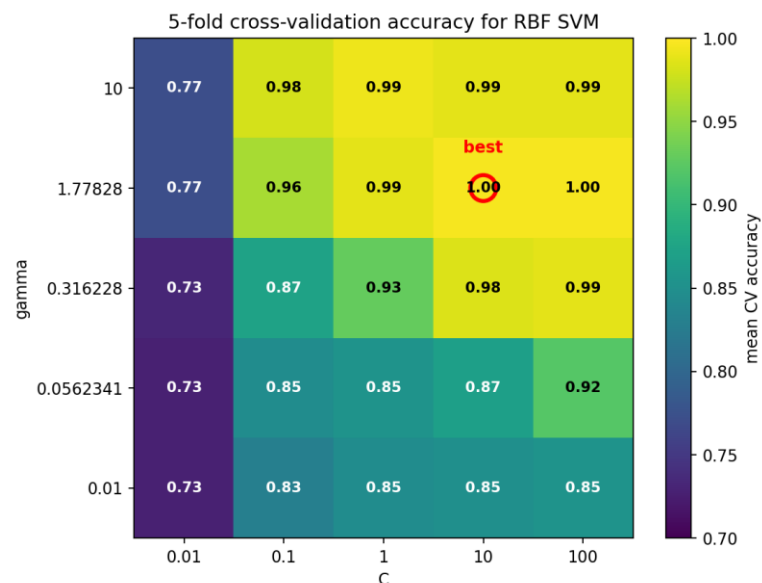


Figure 4: Cross-validation heatmap for RBF SVM hyperparameters. The selected setting is circled.

The cross-validation optimum parameters were found to be $C = 10$ and $\gamma \approx 1.78$, led to mean of cross-validation accuracy of 0.996 and test accuracy of 0.972. This is the more honest final estimate, as the cross-validation score was used during model selection.

Practical checklist and responsible use

Use a straight horizontal line as a baseline. A more complicated kernel may be unnecessary if it works well. Scale the numerical features before fitting an SVM! Tune C and γ cross-validating rather than with the test set. Use colors that are accessible, colours that do not clash with the colour-blind view, use a variety of marker shapes and clear captions (Cramer, Shephard and Heron, 2024). In real-life applications, review the accuracy of subgroups and data bias; accuracy does not ensure fairness or safety (Pessach and Shmueli, 2022).

This tutorial has relied on artificial data, which means that it does not facilitate the demonstration of the suitability of kernel SVMs for any actual task. For applications like medicine, hiring or finance, the performance of the model should be evaluated on the respective groups, recorded and checked by experts before using it.

Conclusion

The kernel trick is possible because the SVM can be expressed in terms of dot products between observations. By substituting those dot products with an appropriate kernel, the model learns as though it were learning a linear separator in a richer feature space but produces a curved separation in its original feature space. The RBF kernel works well on the two-moons data set because the local similarity assumption is correct on this dataset. Practical skill: selecting and tailoring the kernel, rather than just using the most general available model.

References

- Cramer, F., Shephard, G.E. and Heron, P.J. (2024) 'Choosing suitable color palettes for accessible and accurate science figures', *Current Protocols*, 4(8), e1126.
- Ghojogh, B., Ghodsi, A., Karray, F. and Crowley, M. (2021) 'Reproducing kernel Hilbert space, Mercer's theorem, eigenfunctions, Nyström method, and use of kernels in machine learning: tutorial and survey',
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Pessach, D. and Shmueli, E. (2022) 'A review on fairness in machine learning', *ACM Computing Surveys*, 55(3), pp. 1–44.
- Wainer, J. and Fonseca, P. (2021) 'How to tune the RBF SVM hyperparameters? An empirical evaluation of 18 search algorithms', *Artificial Intelligence Review*, 54(6), pp. 4771–4797.
- Wang, Q. (2022) 'Support vector machine algorithm in machine learning', in *2022 IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA)*. IEEE, pp. 750–756.